NVIDIA CUDA vs CPU Computing: A Comparative Analysis of Parallel Processing for High-Performance Data Processing

Jie Ying Goe
Faculty of Computing
Universiti Teknologi Malaysia
Johor Bahru, Malaysia
goeying@graduate.utm.my

***Abstract*—The rapid growth of data-intensive application, particularly in machine learning and artificial intelligence fields, has increased the demand for high-performance data processing. Traditional CPU based computation excels in sequential processing but still facing challenges when large amounts of data are needed to be processed due to limited number of threads and cores.**

This paper investigates NVIDIA CUDA as a parallel computing platform, which utilizes GPU resources to strengthen its performance and scalability for high-performance data processing by improving its ability in general-purpose computing and parallel computing. It also discusses CUDA architecture execution model and memory structure to examine how it supports massively parallel computation. In addition, a comparative study between CPU and CUDA GPU is conducted to highlight the differences between some key aspects and discover the best use case for each type of them.

The findings show CUDA GPU is generally better than CPU, especially in data-parallel workloads, such as deep learning and image processing. Overall, CUDA's ability to accelerate computation makes it plays an important role in modern high-performance environments.

Keywords—CUDA, NVIDIA GPU, parallel computing, GPU programming, high performance computing

I. Introduction

The exponential growth of data-intensive applications in fields such as machine learning, big data analytics and artificial intelligence are increasing and high-performance data processing is needed to be adapted to run this wide variety of data. Before the parallel computing era, traditional computer architectures, primarily on the Central Processing Unit (CPU), were used to process ML and DL models.

According to Gyawali (2023), one major limitation in CPUs is their performance. Each CPU typically has a limited number of cores and restricts the number of threads to be executed simultaneously at a time. As a result, the CPU excels in sequential processing where it can run complex tasks, like complex mathematical calculations. In contrast, GPUs that have thousands of small cores are suitable for deep learning and high volume data processing as they can handle several computation threads in parallel.

However, it becomes increasingly challenging to utilize the GPU resources efficiently becomes increasingly challenging when the data volume and computational complexity continue to expand. To address this, NVIDIA developed CUDA (Compute Unified Device Architecture) in 2007 to provide a programming framework for developers to implement computing on GPUs (Tran et al., 2021). CUDA, as a parallel computing platform and programming model, specializing in massively parallel tasks and completing them in a short time. It is also designed to handle general-computing tasks like CPUs, to improve the performance and scalability of high-performance data processing applications.

This paper aims to investigate the architecture and memory structure of NVIDIA CUDA in high-performance data processing. In addition, the paper discussed how NVIDIA CUDA works in the real world. A comparative analysis is then carried out to find differences between CUDA-based GPU and traditional CPU computing and the last section concludes the paper.

II. NVIDIA CUDA Architecture and Features

A. Overview of CUDA

CUDA is a parallel computing platform and programming model that is designed specifically for NVIDIA GPU devices. There are various types of popular programming languages, such as Python, C++, C, Java and Fortran, that are used in CUDA (Özsoy et al., 2025). This feature provides a highly flexible way for the users while performing GPU programming as both direct first-party and third-party bindings or extensions are supported.

In addition, the CUDA programming model provides structured mechanisms to organize threads and manage access memory on the GPU, enabling efficient utilization of computational resources (Özsoy et al., 2025). By reconstructing a well-defined hierarchy framework, programmers can be able to design and develop parallel applications more efficiently as the execution time is reduced and the performance is improving.

To support this capability, CUDA is built on a specialised architecture that enables large-scale parallelism.

B. Architecture

In CUDA, computation is organized into a hierarchy of basic parts, which include grid, block and thread to utilize the full capability of the graphics card on the system. This hierarchy structure allows GPUs to break down complex problems into smaller, manageable parts. Figure 1 shows an architecture diagram of CUDA.

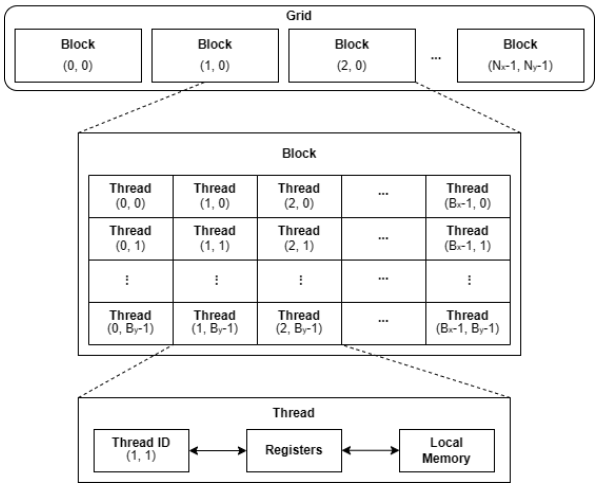

Fig. 1. CUDA Execution Architecture (Ghorpade et al., 2012)

This model contains of three main components :

1) The Grid: Represents the overall problem space processed on the GPU and is the highest level of execution in the model. It is formed by multiple blocks, which are further divided into threads that run together under the same kernel function. For example, in image processing on a 1024 x 1024 pixel image, the grid represents the entire image while each pixel is processed by a thread.

2) The Block: Is a subdivision of the grid and is not shared between GPUs. Each block consists of a group of threads and a certain amount of shared memory, which allow it to execute the block in parallel. Blocks can communicate between each other faster compared to accessing global memory, by using shared memory. However, the independent blocks cannot directly synchronize with other blocks.

3) The Thread: Is the smallest unit of computation in CUDA and is responsible for performing individual computational tasks on different data elements. Each thread has its own thread ID and will be assigned to execute a small task separately. For instance, run a specific portion of codes or execute a particular part of the program. However, since CUDA is designed to handle thousands of threads simultaneously, it can perform parallel computation to manage different tasks together at the same time.
(Li et al., 2025)

C. Memory Hierarchy

In addition to its execution architecture, CUDA employs a hierarchical memory structure to enable efficient data access during parallel computation. Different types of memory (as shown in Fig 2.) are distributed in this memory hierarchy model to handle different functions. Besides, they are designed to maintain the balance between latency, capacity and access speed within the GPU. (Li et al., 2025)

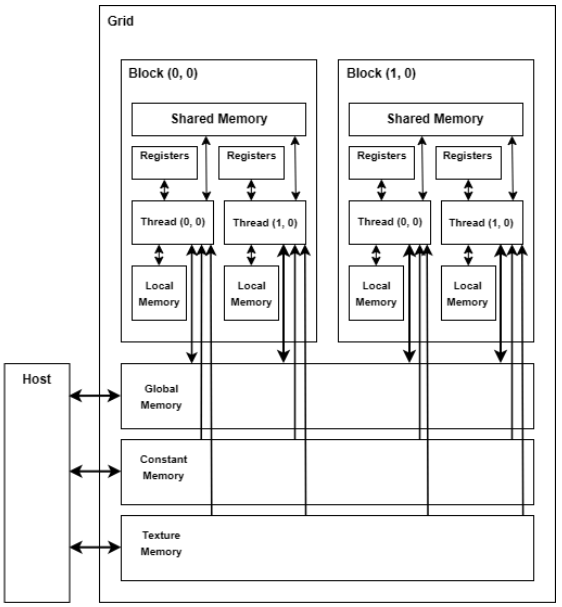

Fig. 2. CUDA Memory Hierarchy (Ghorpade et al., 2012)

The main memory types in CUDA include:

1) Global Memory: a read and write memory. As the main memory space on a GPU, global memory stores large amounts of data as it has a large capacity. All threads have access to global memory and it's shared across many threads. However, it has relatively high latency, causing longer time needed for the users to retrieve the data on it compared to other memory types.

2) Constant Memory: a type of read-only memory that stores constants and kernel arguments (Afif et al., 2020). Unlike global memory, constant memory is cached, which offers high performance when multiple threads access the same data. However, when threads access different memory addresses, requests are split separately into multiple transactions resulting in decreasing throughput and affecting overall performance.

3) Texture Memory: a read only memory. It is read from the kernel and is cached to be optimized for 2D spatial access patterns using texture fetch. When threads access nearby elements, it provides a better performance. Texture memory also offers different addressing modes and can filter the data from specific data formats.

4) Shared Memory: a low latency memory space with smaller size than global memory. All threads in the same block can be executed simultaneously and they share the memory space for read or write operations. Also, collaboration and data exchange within the same block is available. Shared memory is faster to access and during computations, it can be used as a temporary storage. (Li et al., 2025)

5) Registers: the fastest type of memory on the GPU but are limited in number. It is very fast as it only stores variables used by a single thread.

6) Local Memory: resides in device memory and reserved only for threads. It is the same as global memory which is slow and uncached, causing a high latency. Since the number of registers is limited, when the threads need more memory, local memory serves as a backup to store temporary variables that are too large or do not fit into registers.

D. Real-World Application

Meta Platforms operates large scale-social media platforms such as Facebook and Instagram. In a single day, billions of images and videos are needed to be processed. To handle this massive amount of visual data, strong parallel computation for image classification and object detection is crucial to ensure faster response times and high performance processing. Therefore, a machine learning model with CUDA-based computing that can handle large-scale parallel computations is important in this case.

According to (Anderson et al., 2021), Facebook is utilizing Pytorch as their deep learning framework. Pytorch is tightly integrated with NVIDIA CUDA to offload massive matrix operations from CPU to GPU and leverage GPU acceleration. This enables efficient training and inference of deep learning models on user personalized recommendation, image recognition, video and language understanding to make Facebook become a more user friendly platform.

This demonstrates how CUDA plays a critical role in real world high-performance data processing. The large-scale data processing with fast execution provided by CUDA are essential, especially for real time analytics.

III. Comparative Analysis: CPU vs GPU CUDA

TABLE I. COMPARISON BETWEEN CPU AND GPU CUDA (Hasnaine et al., 2025)

Aspect	***CPU***	***GPU with CUDA***
Processing Model	Sequential Processing	Parallel Processing
Architecture Design	Limited Powerful Core (2-64)	Thousand of CUDA and tensor cores
Execution & Thread Model	Few threads per core	Thousands of threads executed simultaneously
Performance	Suitable for sequential, low-latency tasks	10× to 100× speedup in parallel workloads
Memory Hierarchy	L1, L2 and L3 cache (latency-focused)	HBM, GDDR (bandwidth-focused)
Real-world Usage	Database queries, OS administration, HIgh-performance computing	Deep learning, Real-time data processing, Medical Imaging

A. Critical Discussion

Prior to CUDA GPU, CPU was used to perform large-scale training, like deep learning. However, over the last decade, GPUs have undergone improvements, CPUs are facing the bottleneck while processing large numbers of compute-intensive mathematical operations (Kim et al., 2023). It is because CPUs are limited by their small number of cores and only a few threads per core. As a result, it is optimal for sequential processing and general purpose computing but not suitable for high volume workloads that require faster performance through parallel computation.

In contrast, CUDA-enabled GPUs introduced by NVIDIA are equipped with a thousand cores and threads that allow simultaneous executions. GPUs are up to 100 times faster than CPU-only data intensive applications. Although GPUs are initially created for graphics rendering, to perform the same operations simultaneously on many pixels (Hasnaine et al., 2025), they have revolutionized through CUDA, extending their capabilities into general-purpose computing like a CPU does.

Additionally, CPU and CUDA GPU also differ in their memory hierarchy design. CPUs store frequently accessed data using hierarchical caches (L1, L2, L3) to reduce latency and maintain precision significantly. In CUDA GPU architectures, GDDR or HBM provides a high bandwidth memory, which can increase the large-scale data throughput. However, GPUs generally have higher access latency compared to CPU caches (Hasnaine et al., 2025). These differences reflect their usage. For instance, CPU is typically used in control-intensive systems, such as operating systems, while GPU dominates in deep learning and medical imaging that require high throughput through parallel wordloads.

Overall, the comparison shows that CUDA GPUs outperform CPUs in parallel computing due to their architecture design and high memory bandwidth. Despite this, the CPU demonstrates better performance for low-latency, sequential processing. Therefore, it is good to combine both CPU and GPU to achieve better overall performance and efficiency.

IV. Conclusion

In conclusion, this paper examines NVIDIA CUDA architecture and usage in detail. It has transformed GPUs into more powerful parallel computing and programming platforms, which not only implement the key feature of common GPU, like processing large-scale data, but transformed GPUs into general computing platforms. It provides a better choice with improved performance and scalability for developers.

References

Gyawali, D. (2023). Comparative analysis of CPU and GPU profiling for deep learning models. arXiv. https://doi.org/10.48550/arXiv.2309.02521

Tran, B. Q., Nguyen, T. V., Tran, D. D., Tran, A. D., & Nguyen, H. V. (2021). Accelerating exemplar-based image inpainting with GPU and CUDA. In ACM International Conference Proceeding Series (pp. 173–179). https://doi.org/10.1145/3457784.3457812

Özsoy, A., Nazlı, M., Cankur, O., et al. (2025). CUSMART: Effective parallelization of string matching algorithms using GPGPU accelerators. Frontiers of Information Technology & Electronic Engineering, 26, 877–895. https://doi.org/10.1631/FITEE.2400091

Li, M., Bi, Z., Wang, T., Wen, Y., Niu, Q., Song, X., Jiang, Z., Liu, J., Peng, B., Zhang, S., Pan, X., Xu, J., Wang, J., Chen, K., Yin, C. H., Feng, P., & Liu, M. (2025). *Deep learning and machine learning with GPGPU and CUDA: Unlocking the power of parallel computing* (Version 3). arXiv. https://doi.org/10.48550/arXiv.2410.05686

Ghorpade, J., Parande, J., Kulkarni, M., & Bawaskar, A. (2012). *GPGPU processing in CUDA architecture*. arXiv. https://doi.org/10.48550/arXiv.1202.4347

Afif, M., Said, Y., & Atri, M. (2020). *Computer vision algorithms acceleration using graphic processors NVIDIA CUDA*. *Cluster Computing, 23*, 3335–3347. https://doi.org/10.1007/s10586-020-03090-6

Anderson, M., Chen, B., Chen, S., Deng, S., Fix, J., Gschwind, M., et al. (2021). First-generation inference accelerator deployment at Facebook. arXiv. https://doi.org/10.48550/arXiv.2107.04140

Hasnaine, Q. R., Fouda, M. M., & Chiu, S. C. (2025). GPU vs. CPU: A comparative study of performance, architecture, and NVIDIA's innovations in modern computing. In 2025 IEEE 4th International Conference on Computing and Machine Intelligence (ICMI) (pp. 1–6). IEEE. https://doi.org/10.1109/ICMI65310.2025.11141315

Kim, T., Park, C., Mukimbekov, M., Hong, H., Kim, M., Jin, Z., Kim, C., Shin, J. Y., & Jeon, M. (2023). FusionFlow: Accelerating data preprocessing for machine learning with CPU–GPU cooperation. Proceedings of the VLDB Endowment, 17(4), 863–876. https://doi.org/10.14778/3636218.3636238